



University of
Nottingham

UK | CHINA | MALAYSIA

A large, high-resolution image of the Earth as seen from space, showing the Western Hemisphere with North and South America. The image is framed by a thin white border.

COMP-2024 / COMP-2039
**Artificial Intelligence
Methods**

Lecture 5
Evolutionary Algorithms I



Learning Outcomes

IDENTIFY

1. Evolutionary Algorithms
2. Genetic Algorithms (GAs)
3. Memetic Algorithms
4. Benchmark Functions



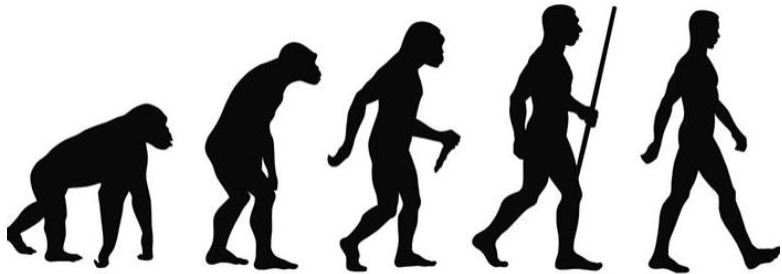
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



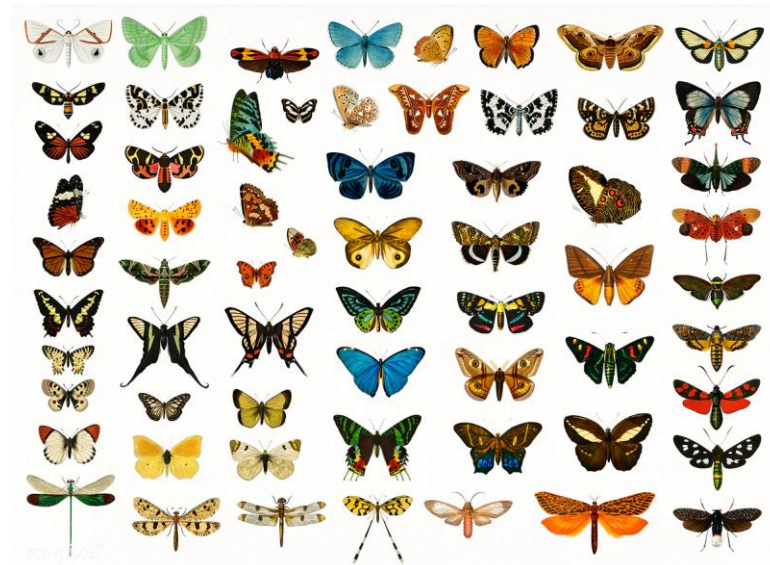
Evolutionary is Revolutionary

- **Nature as a Problem Solver:** 4.55 Billion years of evolution can't be wrong.
- **Beauty-of-nature argument:** Complexity achieved in *short* time in nature.
- **Can we solve complex problems as quickly and reliably on a computer?**

Fine-Tuning and Order: The complexity and harmony of nature suggest intentional design rather than randomness.



Evolution: Gradual change in the inherited characteristics of a population of animals or plants over successive generations



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Evolution at Work

- Heritable characteristics or heritable traits, e.g., the colour of your eyes are passed from one generation to the next via DNA (a molecule that encodes genetic information)
- Change or genetic variation comes from:
 - **Mutations**: changes in the DNA sequence,
 - **Crossover**: reshuffling of genes through sexual reproduction and migration between populations
- **Evolution** is driven by *natural selection – survival of the fittest*
- Genetic variations that enhance survival and reproduction become and remain more common in successive generations of a population



A famous example – peppered moth evolution

- The example of peppered moth evolution is a classic case study in natural selection and illustrates how environmental changes can drive evolutionary adaptations in species.
- Before the Industrial Revolution, the black peppered moth was rare.
 - The frequency of the dark allele was about 0.01%
- The rapid industrialisation and rampant coal use coated the British trees in a layer of soot
- By the mid-19th century, the number of dark-coloured moths had risen noticeably, and by 1895, fr. was 98% in Manchester



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Types of Evolutionary Algorithms

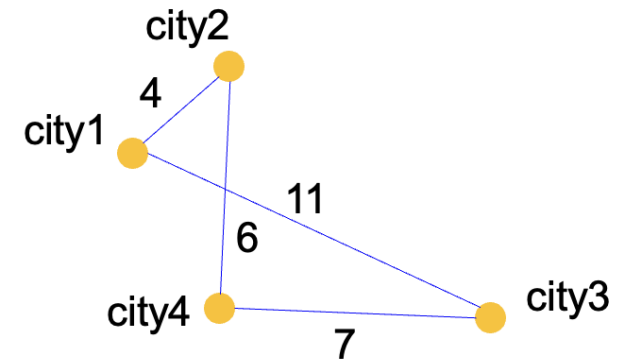
- **Genetic Algorithms** (evolves (bit) strings) $\Rightarrow$ Nils Aall Barricelli 1954, Holland 1975
 - **Memetic Algorithms** $\Rightarrow$ Moscato 1989
- **Evolutionary Programming** (evolves parameters of a program with a fixed structure) $\Rightarrow$ Fogel, Owens, Walsh 1966
- **Evolution Strategies** (vectors of real numbers) $\Rightarrow$ Rechenberg 1973
- **Genetic Programming** (evolves computer programs in tree form) $\Rightarrow$ Koza 1992
 - **Gene Expression Programming** (computer programs of different sizes are encoded in linear chromosomes of fixed length)
 - **Grammatical Evolution** (evolves solutions wrt a specified grammar) $\Rightarrow$ Ryan, Collins and O'Neill 1998

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Evolutionary Algorithms (EAs)

- GAs (John Holland) simulate natural evolution (Darwinian Evolution) of **individual** structures at the genetic level using the idea of **survival of the fittest** via processes of **selection, mutation, and reproduction (recombination)**
- An **individual (chromosome)** represents a candidate solution for the problem at hand (e.g., $\langle 2 \ 1 \ 3 \ 4 \rangle$)
- A collection of individuals currently “alive”, called **population** is evolved from one **generation** to another depending on the **fitness** of individuals in a given environment, indicating how fit an individual is, in other words, how close it is to the optimal solution – objective value. (e.g., $f(\langle 2 \ 1 \ 3 \ 4 \rangle) = 28$)
- **Hope:** Last generation will contain the final solution



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



EAs – Features

- **Avoid converging to local optima**
- Use *exploration* of the search space and *exploitation* of promising areas.
 - **Reproduction** focuses attention on high fitness individuals, thus *exploiting* the available fitness information.
 - **Recombination (crossover)** and **mutation** perturb those individuals, providing general heuristics for *exploration*.
Through recombination operators, reach better solutions by combining already found good solutions



EAs – Features II

- Be independent of initial starting point(s)
 - Start search from many points in the search space
- Conduct search in *parallel* over the search space
 - Implicit parallelism
- May be used together with other approaches (hybrids)
 - Memetic Algorithm: Genetic + Local Search



University of
Nottingham

UK | CHINA | MALAYSIA

Genetic Algorithm (GA)



How Genetic Algorithms Work

1. **Initialisation:** Start with a population of candidate solutions.
2. **Evaluate** each solution's fitness (how good it is).
3. **Select** the best solutions to create offspring.
4. Apply **crossover** and **mutation** to generate new solutions.
5. **Replace** the old population with the new one and repeat until a stopping criterion is met (e.g., a maximum number of generations or a satisfactory solution).



Pseudocode of a Generic Genetic Algorithm

```
begin
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
perform reproduction; // select parents
  recombine pairs with  $P_c$ ; // apply crossover
  apply mutation with  $P_m$ ; // mutate offspring/children
calculate fitness values; // eval. population
replace current population;
} while termination criteria not satisfied;
end
```

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Core Concepts of Genetic Algorithms

■ 1. Representation (Encoding)

- A solution is represented as a **chromosome** (string of values).
- Common encodings:
 - **Binary Encoding:** Chromosomes are binary strings (e.g., "101011").
 - **Permutation Encoding:** Used for ordering problems like TSP (e.g., [A, C, B, D]).
 - **Real-Value Encoding:** Chromosomes are vectors of real numbers (e.g., [2.5, 1.7, 3.8]).



Core Concepts of Genetic Algorithms

■ 2. Fitness Function

- The fitness function measures how "good" a solution is.
- Example: For maximizing $f(x)=x^2$, fitness for $x=3$ is $f(3)=9$.



Core Concepts of Genetic Algorithms

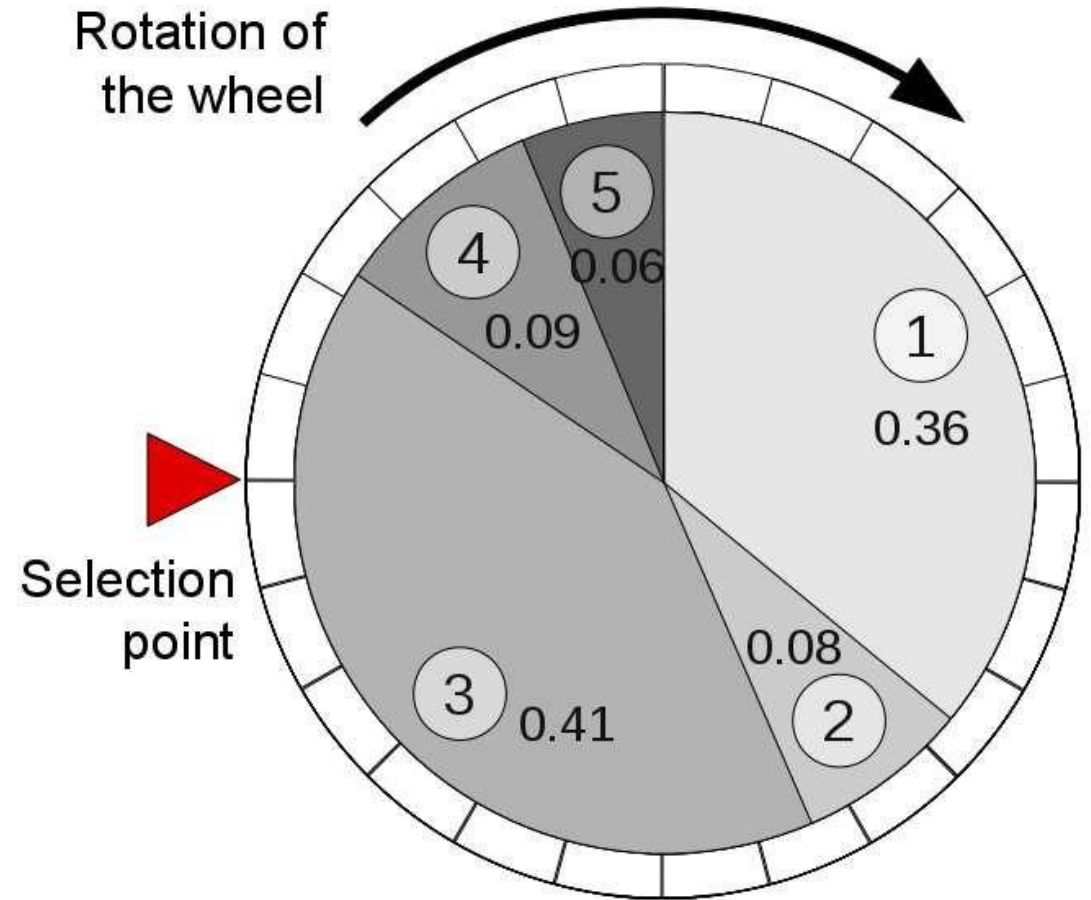
■ 3. Selection

- Select individuals (parents) for reproduction based on their fitness.
- Methods:
 - **Roulette Wheel Selection:** Probability of selection is proportional to fitness.
 - **Tournament Selection:** Randomly pick a group and select the best.
 - **Rank Selection:** Rank individuals and assign probabilities based on rank.



Roulette Wheel Selection

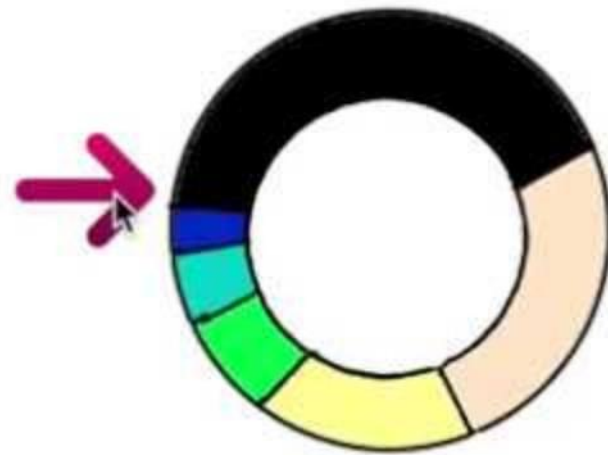
- This method involves assigning a probability of selection based on an individual's fitness relative to the total fitness of the population.
- **Mechanism:** Imagine a roulette wheel divided into sections; each section's size corresponds to an individual's fitness. When the wheel is spun, individuals with higher fitness are more likely to be selected.



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Roulette Wheel Selection



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

<https://www.youtube.com/watch?v=rrJSluytaQU>



Roulette Wheel Selection

Permutation Genetic Algorithms

The Roulette Wheel Selection Method

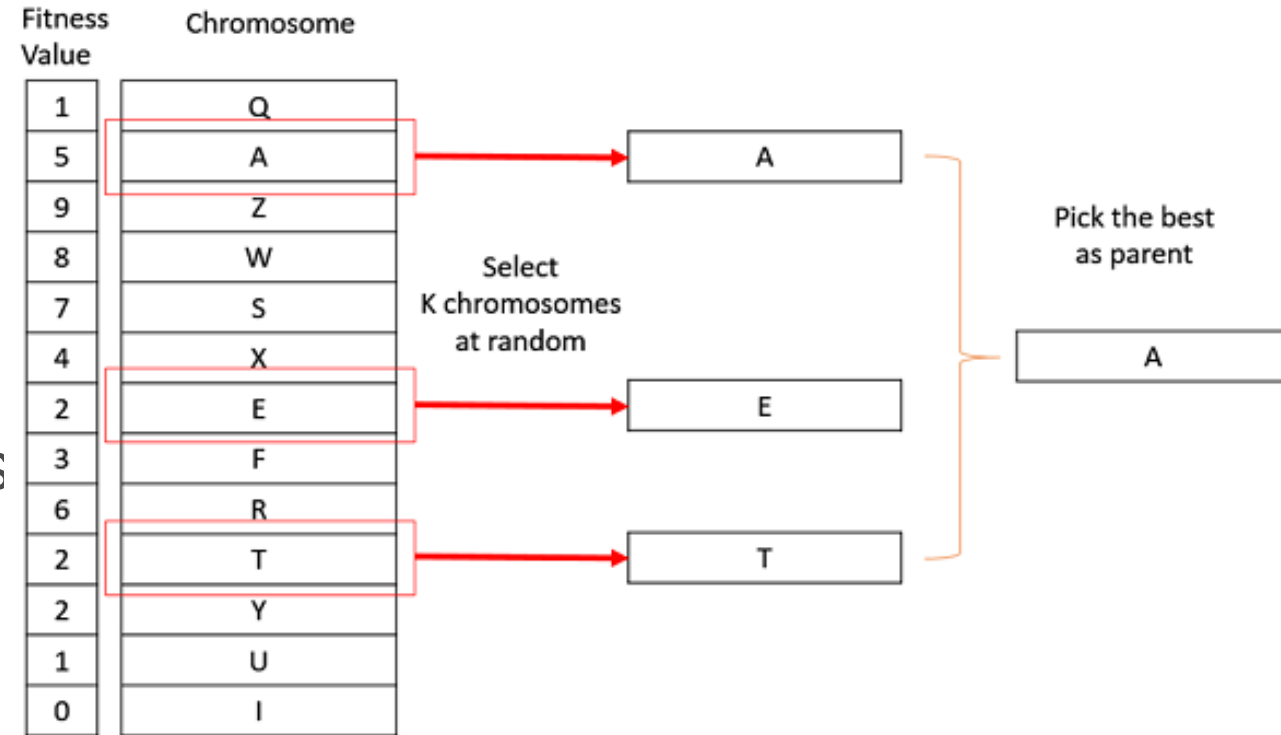
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

<https://www.youtube.com/watch?v=qMUaGYcOF2g>



Tournament Selection

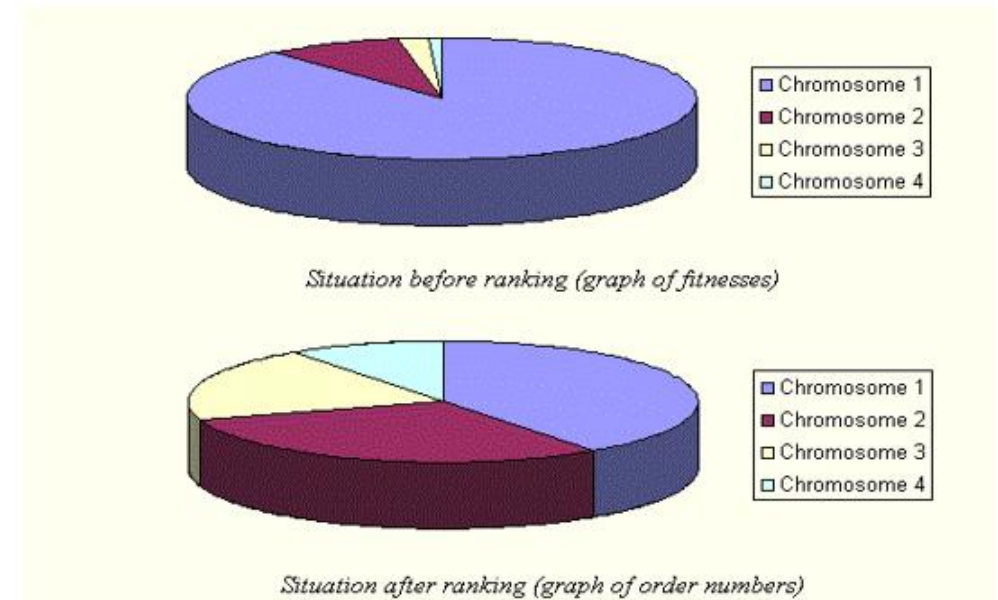
- This method involves randomly selecting a subset of individuals from the population and choosing the best one among them.
- **Mechanism:** A fixed number of individuals are *chosen at random*, and the one with the highest fitness in that group is selected for reproduction. This method can create a *balance between exploration and exploitation*.





Rank Selection

- In this method, individuals are ranked based on their fitness, and *selection probabilities are assigned according to their rank*, rather than their absolute fitness values.
- **Mechanism:** This helps prevent premature convergence by ensuring that even lower-ranked individuals have a chance to be selected (prevent dominance), fostering diversity in the population.



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



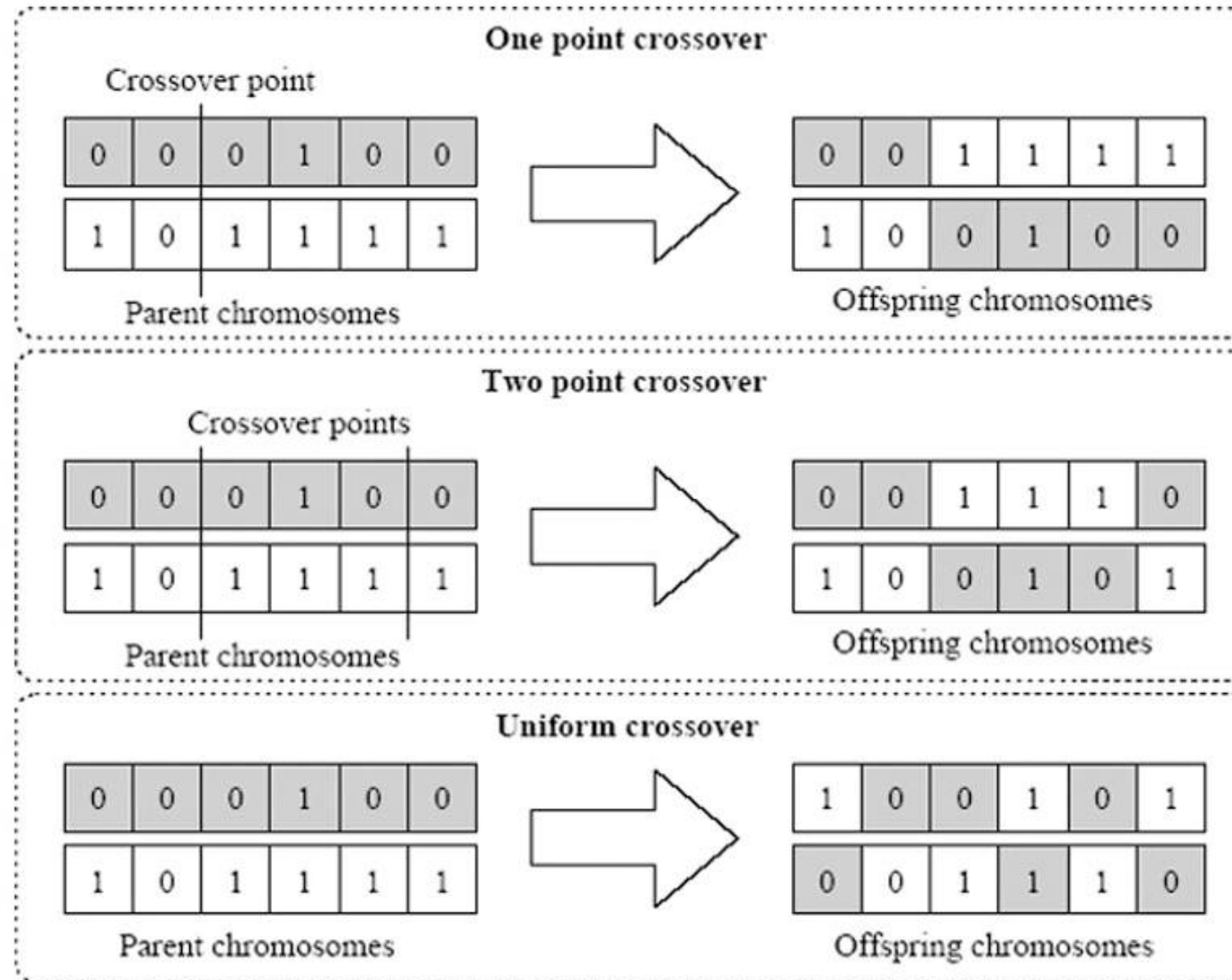
Core Concepts of Genetic Algorithms

■ 4. Crossover (Recombination)

- Combine two parent solutions to create offspring.
- Methods:
 - **Single-Point Crossover:** Split at one point and exchange segments.
 - **Two-Point Crossover:** Split at two points and swap middle segments.
 - **Uniform Crossover:** Mix genes randomly.



Crossover (Recombination)



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul &



Core Concepts of Genetic Algorithms

■ 5. Mutation

- Introduce *small random changes* to maintain diversity (for exploration).
- Examples:
 - **Bit Flip (Binary)**: Flip a bit in the chromosome (e.g., "101" → "111").
 - **Swap (Permutation)**: Swap two genes (e.g., [A, B, C] → [A, C, B]).



Core Concepts of Genetic Algorithms

■ 6. Replacement

- “**Replacement strategy**” determines which individuals from the current population (including parents and offspring) will be chosen to survive and become part of the next generation, essentially deciding which solutions are kept and which are discarded
- Strategies:
 - Replace the whole population.
 - Replace only the weakest individuals.



Replacement Strategy

■ Generational Replacement

- The entire population is replaced by the offspring in each generation.
- Suitable for problems where *diversity* needs to be maintained.
- Can lead to rapid evolution but may suffer from *premature convergence*.

■ Steady-State Replacement

- Only a few individuals (often one or two) are replaced per iteration.
- Maintains more diversity as some parents survive into the next generation.
- Typically uses **($\mu + \lambda$) selection**, where μ parents and λ offspring compete.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Replacement Strategy

■ Elitism (Elitist Replacement)

- A subset of the best individuals from the current generation (parents) is carried forward unchanged.
- Prevents loss of the best solutions found so far.
- Often combined with other strategies like generational replacement.

■ Tournament-Based Replacement

- A tournament is held between individuals (e.g., parents vs. offspring).
- Winners of the tournament survive to the next generation.
- Helps balance exploration and exploitation.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



In summary: Basic Components of GAs

- A genetic **representation (encoding)** for candidate solutions to the problem at hand
- An **initialisation** scheme to generate the first population of candidate solutions
- A **fitness function** that plays the role of the environment, rating the solutions in terms of their fitness
- A scheme for **selecting mates (parents)** for recombination
- **Crossover (recombination)**
- **Mutation**
- **Replacement strategy** to select the surviving individuals for the next generation
- **Termination Criteria**
- Values for various **parameters** that the genetic algorithm uses (population size, probabilities of applying genetic operators, etc.)

ACK: Prof. Ender Özcan, Univ. of York, Dr. Thomas Martin & Dr. Omer Zekiye, UNM



University of
Nottingham

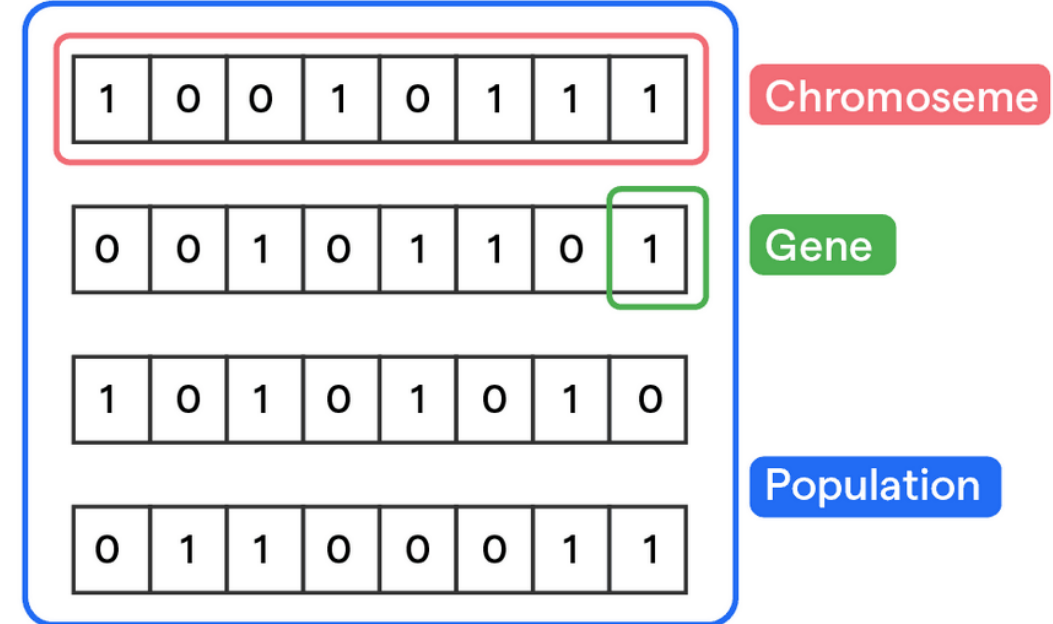
UK | CHINA | MALAYSIA

GA Example: MAX-SAT



GA Components: Representation

- Haploid structure is used: Each individual contains one chromosome
- Each individual is evaluated and has an associated **fitness** value
- Chromosomes contain a fixed number of genes: **chromosome length**
- Traditionally **binary encoding** is used for each gene: **Allele** value $\in \{0,1\}$
- A population contains a fixed number of individuals: **population size**
- Each iteration is referred as **generation**



ACK: Prof. Guo-Qizhan, UNHK, Dr. Jia-Hui Mao & Dr. Chen-Zhi Yuan, JNIM

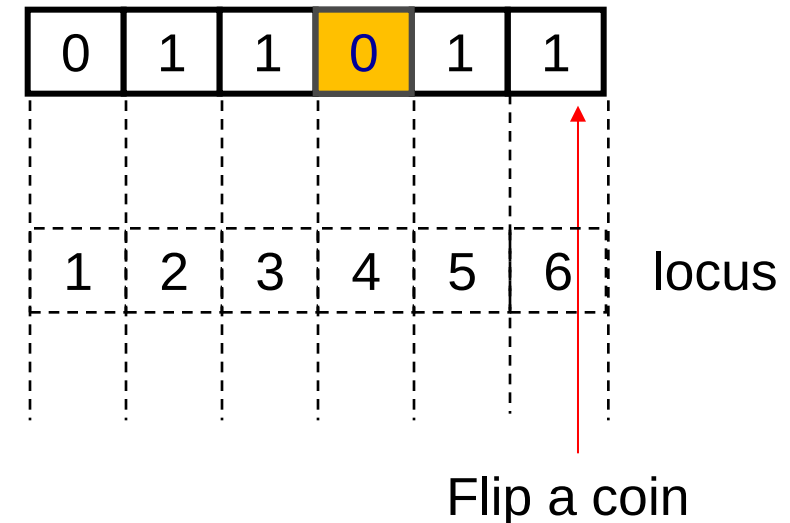


GA Components: **Initialisation**

- Random Initialisation
- **Population size** - number of individuals are created randomly
- Each gene at a locus of an individual is assigned an **allele** value 0 or 1 randomly, decided by flipping a coin. (E.g., if the random value is < 0.5 , then allele is assigned to 0, otherwise to 1)

Chromosome (individual)

gene





Example – MAX-SAT: Initialisation

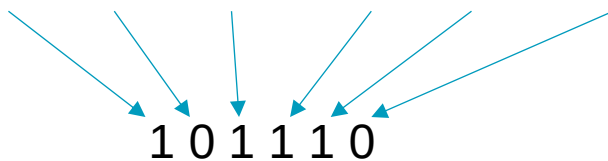
$$\begin{array}{cccccc} 0 & & 1 & & 2 & & 3 & & 4 & & 5 \\ (a \vee b) & \wedge & (\neg d \vee f) & \wedge & (\neg a \vee c) & \wedge & (b \vee \neg f) & \wedge & (\neg b \vee c) & \wedge & (c \vee e) \end{array}$$

1: <0.98, 0.85, 0.13, 0.04, 0.47, 0.44>

2: <0.09, 0.63, 0.22, 0.54, 0.07, 0.37>

3: <0.21, 0.03, 0.72, 0.84, 0.19, 0.49>

4: <0.61, 0.43, 0.87, 0.53, 0.97, 0.14>



- Assume *population size* is 4
- The individual size/*chromosome length* is 6 (since we have 6 literals: *abcdef*)
- So, create 4 individuals with 6 genes within their chromosomes, where each allele at a locus is determined **randomly** (by throwing a random number in $[0,1)$) $\rightarrow$ 0 if < 0.5 and 1 if ≥ 0.5



Example – MAX-SAT: Initialisation

$$\begin{array}{cccccc} 0 & & 1 & & 2 & & 3 & & 4 & & 5 \\ (a \vee b) & \wedge & (\neg d \vee f) & \wedge & (\neg a \vee c) & \wedge & (b \vee \neg f) & \wedge & (\neg b \vee c) & \wedge & (c \vee e) \end{array}$$

<i>i</i>	Chromosome <i>abcdef</i>
1:	110000
2:	010100
3:	001100
4:	101110

- Assume *population size* is 4
- The individual size/*chromosome length* is 6 (since we have 6 literals: *abcdef*)
- So, create 4 individuals with 6 genes within their chromosomes, where each allele at a locus is determined **randomly** (by throwing a random number in $[0,1)$).



Generic GA – Fitness Calculation

- **Fitness value** (objective function value) indicates
 - how fit the individual is to survive and reproduce under the current conditions
 - how much the current solution meets the requirements of the objective function
- **Fitness value** is obtained by applying the fitness function to the individual's *chromosome / genotype* (candidate solution) (e.g., 101110) to *phenotype* (e.g., 1) mapping
 - **Genotype**: This refers to the actual representation of the chromosome in the algorithm. It is the underlying genetic code (e.g., 101110).
 - **Phenotype**: This is the actual expression of the genotype in the context of the problem. It is the observable outcome or solution derived from the genotype (e.g., 1).

ACK: Prof. Ender Ozcan, UNUK, Dr Thomas Maul & Dr Chen ZhiYuan, UNM



Example – MAX-SAT: Fitness Calculation

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ (a \vee b) & \wedge & (\neg d \vee f) & \wedge & (\neg a \vee c) & \wedge & (b \vee \neg f) & \wedge & (\neg b \vee c) & \wedge & (c \vee e) \end{array}$$

<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>
1:	110000	012345	3
2:	010100	012345	3
3:	001100	012345	2
4:	101110	012345	1

```

begin
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
perform reproduction; // select parents
recombine pairs with  $P_c$ ; // apply crossover
apply mutation with  $P_m$ ; // mutate offspring/children
calculate fitness values; // eval. population
replace current population;
} while termination criteria not satisfied;
end

```



GA Components - Reproduction

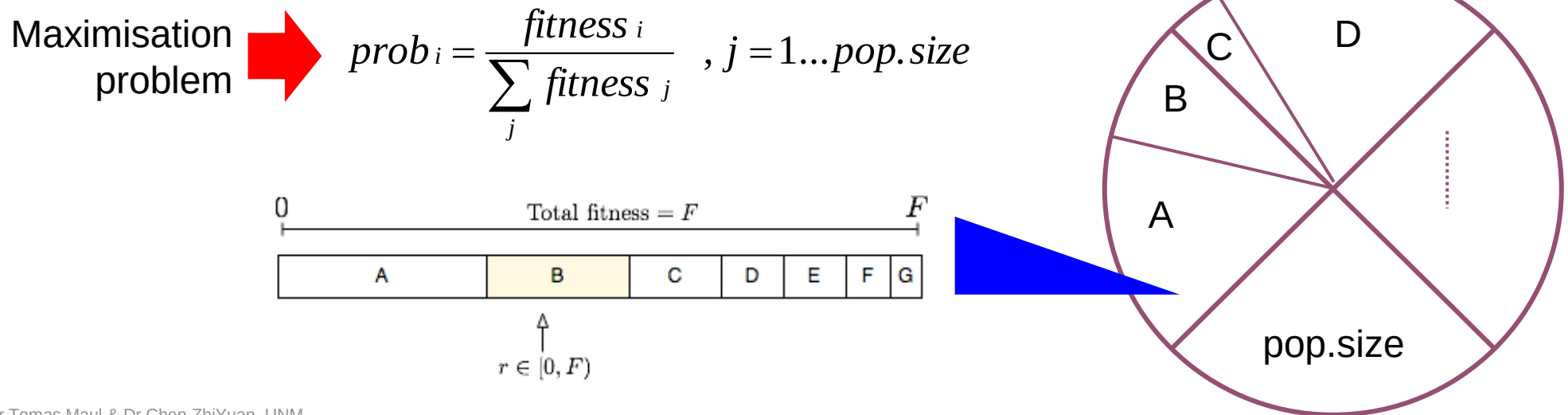
- **Reproduction** (Mate Selection) consists of
 - **selecting** individuals: apply selection pressure considering the fitness of individuals in the population $\Rightarrow$ e.g., *roulette wheel selection*, *tournament selection*, rank selection, truncation selection, Boltzmann selection, etc.
 - ***Selection pressure*** means the individuals with better fitness have higher chance for being selected
 - Usually **2 parents** (individuals/candidate solutions) are selected using the same method, which will go under the crossover operation

```
begin
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
perform reproduction; // select parents
recombine pairs with  $P_c$ ; // apply crossover
apply mutation with  $P_m$ ; // mutate offspring/children
calculate fitness values; // eval. population
replace current population;
} while termination criteria not satisfied;
end
```



Fitness Proportionate Selection – Roulette Wheel Selection

- Fitness level is used to associate a **probability** ($prob_i$) of selection with each individual chromosome (i)
- While candidate solutions with a higher fitness will be less likely to be eliminated, there is still a chance that they may be
- Expected number of representatives of each individual in the pool is *proportional to its fitness*



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

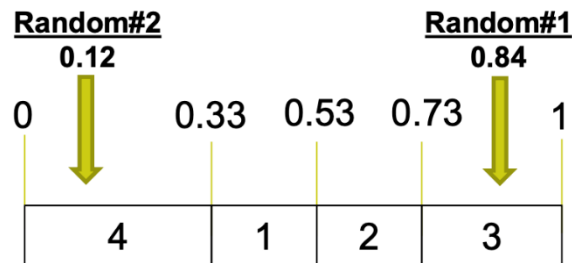


Example – MAX-SAT: Roulette Wheel Selection

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \\ (a \vee b) & \wedge & (\neg d \vee f) & \wedge & (\neg a \vee c) & \wedge & (b \vee \neg f) & \wedge & (\neg b \vee c) & \wedge & (c \vee e) \end{array}$$

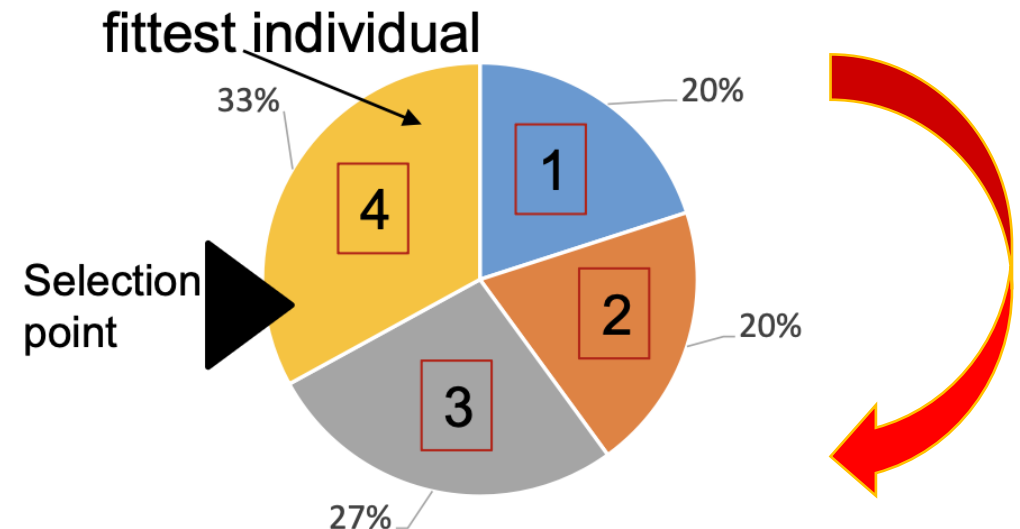
<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>	<i>prob_i</i>
1:	110000	01 2 345	3 (<u>3</u>)	20%
2:	010100	01 2 345	3 (<u>3</u>)	20%
3:	001100	0 12345	2 (<u>4</u>)	27%
4:	101110	01 2 345	1 (<u>5</u>)	33%

total: (15)



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

- Rotate the Wheel



Random#1: 0.84 → Parent#1: 3

Random#2: 0.12 → Parent#2: 4



Tournament Selection

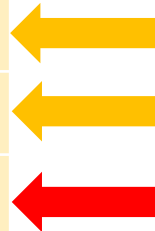
- This method involves running a number of "tournaments" among randomly chosen individuals (of tour size) selecting the one with **best fitness** at the end
 - This process is repeated for selecting each parent to be recombined



Example – MAX-SAT: Tournament Selection

tour size = 3, first parent

<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>
1:	110000	012345	3
2:	010100	012345	3
3:	001100	012345	2
4:	101110	012345	1



```
begin
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
perform reproduction; // select parents
recombine pairs with  $P_c$ ; // apply crossover
apply mutation with  $P_m$ ; // mutate offspring/children
calculate fitness values; // eval. population
replace current population;
} while termination criteria not satisfied;
end
```

- Throw a random number between 1 and 4 (population size) for **3** times:
 - [3, 1, 2]
- Tournament selection chooses 3 individuals: #1, #2 and #3 at random, then individual#3 with the fitness of 2 is returned as the first parent (minimization)



Example – MAX-SAT: Tournament Selection

tour size = 3, second parent

<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>	
1:	110000	012345	3	←
2:	010100	012345	3	
3:	001100	012345	2	←
4:	101110	012345	1	←

```
begin
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
perform reproduction; // select parents
recombine pairs with  $P_c$ ; // apply crossover
apply mutation with  $P_m$ ; // mutate offspring/children
calculate fitness values; // eval. population
replace current population;
} while termination criteria not satisfied;
end
```

- Throw a random number between 1 and 4 (population size) for **3** times:
 - [4, 1, 3]
- Tournament selection chooses 3 individuals: #1, #3 and #4 at random, then individual#4 with the fitness of 1 is returned as the second parent (minimization)



Other Selection Methods

■ Rank Selection

- Individuals are *sorted based on fitness and assigned ranks*.
- Selection probability is based on *rank* rather than absolute fitness.
- Ensures that even individuals with low fitness have a fair selection probability.
- Pros: Prevents domination by highly fit individuals; ensures steady selection pressure.
- Cons: Slower convergence compared to fitness-proportionate methods.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Recombination - Crossover

- Selected pairs/mates (*parents*) are **recombined** to form new individuals (candidate solutions/children/offspring) – exchange of genetic material
- Crossover is applied with a **crossover probability** p_c which in general is chosen close to 1.0

One Point Crossover (1PTX)

- Generate a random number in $[0,1)$, if it is smaller than a **crossover probability** p_c Then
 - Select a random crossover site in $[1, \text{chromosome length}]$
 - Split individuals at the selected site
 - Exchange segments between pairs to form two new individuals
- Else
 - Copy the individuals as new individuals

```
begin
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
perform reproduction; // select parents
recombine pairs with  $P_c$ ; // apply crossover
apply mutation with  $P_m$ ; // mutate offspring/children
calculate fitness values; // eval. population
replace current population;
} while termination criteria not satisfied;
end
```



Example – MAX-SAT: 1PTX

0 1 2 3 4 5
 $(a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e)$

Random
number: 2

Randomly determined
crossover point

0 0 1 1 0 0

1 0 1 1 1 0

1 0 1 1 0 0

0 0 1 1 1 0

1. Throw a random
number in [1..6]

new solution/children/offspring

```
begin
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
perform reproduction; // select parents
recombine pairs with  $P_c$ ; // apply crossover
apply mutation with  $P_m$ ; // mutate offspring/children
calculate fitness values; // eval. population
replace current population;
} while termination criteria not satisfied;
end
```

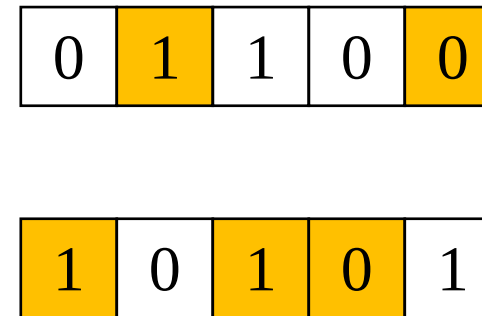
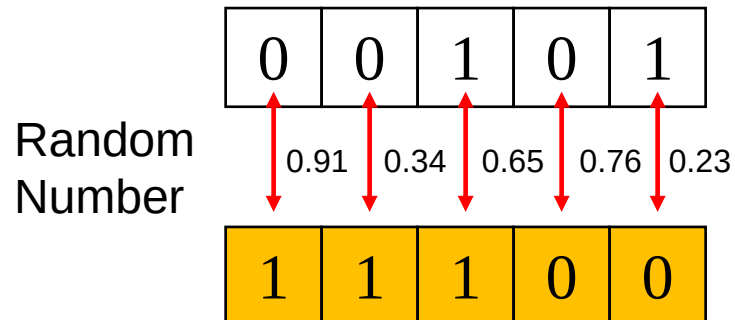
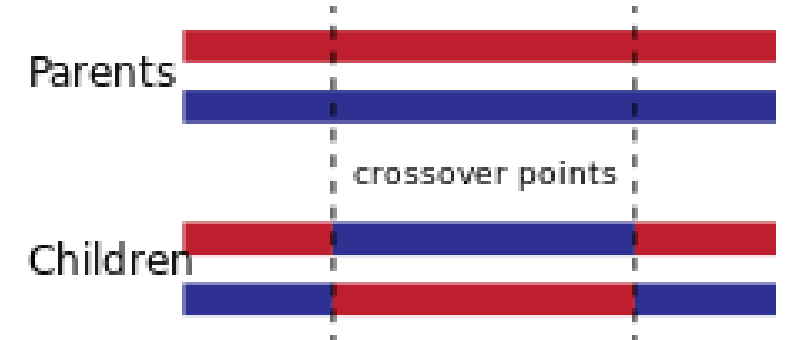
2. Exchange the genetic
material



Other Crossover Operators

- 2 Point Crossover (2PTX)
- K-point Crossover
- Uniform Crossover (UX)

- The uniform crossover considers each bit in the parent strings for exchange with a probability of 0.5.





Mutation

- Any offspring might be exposed to **mutation**
- Loop through all the alleles of all the individuals one by one, and if that allele is selected for mutation with a given probability p_m , you can either change it by a small amount or replace it with a new value
 - For a **binary** representation mutation corresponds to **flipping** a selected gene value (0→1, 1→0)
- Mutation provides *diversity* and allows GA to investigate different regions of the search space (escaping local minima)
- *Mutation rate* is typically chosen to be very small (e.g. 0.001). Choosing p_m as **(1/chromosome length)** implies on average a single gene will be mutated for an individual.



Example - Mutation

1	0	1	1	0	0
---	---	---	---	---	---

- A loop is performed on each individual
 - If a random value in $[0,1)$ is $< p_m = 0.2$ (1/5), then the allele value is flipped, otherwise kept same.

0	0	1	1	1	0
---	---	---	---	---	---

```
begin
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
perform reproduction; // select parents
recombine pairs with  $P_c$ ; // apply crossover
apply mutation with  $P_m$ ; // mutate offspring/children
calculate fitness values; // eval. population
replace current population;
} while termination criteria not satisfied;
end
```



Example - Mutation

Random numbers:

<0.23 0.76 0.41 0.14 0.91 0.68 0.47 0.19 0.28 0.94 0.78 0.03 ...>



0.23 0.76 0.41 0.14 0.91 0.68

1	0	1	0	0	0
---	---	---	---	---	---



0.47 0.19 0.28 0.94 0.78 0.03

0	1	1	1	1	1
---	---	---	---	---	---

```
begin
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
perform reproduction; // select parents
recombine pairs with  $P_c$ ; // apply crossover
apply mutation with  $P_m$ ; // mutate offspring/children
calculate fitness values; // eval. population
replace current population;
} while termination criteria not satisfied;
end
```

- A loop is performed on each individual
 - If a random value in $[0,1)$ is $< p_m = 0.2$ (1/5), then the allele value is flipped, otherwise kept same.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Replacement Strategy

- There are variety of strategies for replacing the old population (generation) by the new (offspring) population to form the next generation
- **Generation gap (α)** controls the fraction of the population to be replaced in each generation, where $\alpha \in [1/N, 1.0]$
 - Number of offspring produced at each generation is $g = \alpha * N$
- **(Trans-) Generational GA** ($g > 2$, that is $\alpha > 2/N$) N individuals produce αN offspring, so $(N + \alpha N) \rightarrow N$
 - αN replaces worst αN of N
 - largest generation gap where $\alpha = 1.0$ yields $g = N$.
 - GA relies on improvement of average objective values from one population to another
 - It is always a good idea not to loose the best solution found so far.
 - sort $(N + \alpha N)$ and choose the N best (elitism)

```
begin
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
perform reproduction; // select parents
recombine pairs with  $P_c$ ; // apply crossover
apply mutation with  $P_m$ ; // mutate offspring/children
calculate fitness values; // eval. population
replace current population;
} while termination criteria not satisfied;
end
```



Generation Gap (α)

- The **generation gap** α represents the fraction of the population N that is replaced in each generation.
- It ranges from $\alpha \in [1/N, 1.0]$, meaning at least one individual must be replaced, but up to the entire population can be replaced.
- The number of offspring produced at each generation, g is given by:
 - $g = \alpha \times N \rightarrow \alpha = g/N$
 - where:
 - $\alpha = 1/N \rightarrow$ Only one individual is replaced.
 - $\alpha = 1.0 \rightarrow$ The entire population is replaced (generational replacement).



Trans-Generational GA ($\alpha > 2/N$)

- When $\alpha > 2/N$, the algorithm generates more than 2 offspring per parent.
 - The population replacement strategy is as follows:
 - N individuals produce αN offspring.
 - The combined pool now has $(N + \alpha N)$ individuals.
 - The worst αN individuals are removed, keeping only N best individuals for the next generation.
- This ensures that the best solutions persist while introducing enough variation for improvement.



Example – Transgenerational GA Replacement (no elitism)

$$0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5$$
$$(a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e)$$

<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>
1:	100100	012345	3
2:	010100	012345	3
3:	101000	012345	0
4:	011111	012345	0

New
Population/Generation

Form the new
generation by

←
Copying offspring
onto the old
population,
forming the new
population

$$g=N=4$$

<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>
1:	100100	012345	3
2:	010100	012345	3
3:	101000	012345	0
4:	011111	012345	0

Offspring

<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>
1:	110000	012345	3
2:	010100	012345	3
3:	001100	012345	2
4:	101110	012345	1

Old Population

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Example – Transgenerational GA Replacement (with elitism)

$$0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5$$

$$(a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e)$$

<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>
1:	001100	012345	2
2:	101110	012345	1
3:	101000	012345	0
4:	011111	012345	0

New
Population/Generation

Form the new
generation by

Selecting the
best 4 individuals
among both old
population and
offspring

$$g=N=4$$

<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>
1:	100100	012345	3
2:	010100	012345	3
3:	101000	012345	0
4:	011111	012345	0

Offspring

<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>
1:	110000	012345	3
2:	010100	012345	3
3:	001100	012345	2
4:	101110	012345	1

Old Population

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Steady-State Replacement

- A few individuals (typically one or two) are replaced per iteration instead of the whole population.
- The worst-performing individuals are removed, and new offspring replace them.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Replacement Strategy

Steady-State GA ($g=2$, that is $\alpha=2/N$)

Two offspring replace two individuals from the old generation.

- **Method#1:** two offspring replace two parents
- **Method#2:** two offspring replace worst two of the population
- **Method#3:** best two of (parents and offspring) replace two parents (elitism)
- **Method#4:** best two of (parents and offspring) replace worst two of the population (strong elitism)

```
begin
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
perform reproduction; // select parents
recombine pairs with  $P_c$ ; // apply crossover
apply mutation with  $P_m$ ; // mutate offspring/children
calculate fitness values; // eval. population
replace current population;
} while termination criteria not satisfied;
end
```

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Example – a Steady State GA (with elitism)

$$\begin{array}{cccccc} 0 & & 1 & & 2 & & 3 & & 4 & & 5 \\ (a \vee b) & \wedge & (\neg d \vee f) & \wedge & (\neg a \vee c) & \wedge & (b \vee \neg f) & \wedge & (\neg b \vee c) & \wedge & (c \vee e) \end{array}$$

<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>
1:	010100	012345	3
2:	101000	012345	0
3:	001100	012345	2
4:	101110	012345	1

New
Population/Generation

Form the new
generation by

Using Method#2:
Replace the
worst 2 of the
population with 2
offspring

$g=2$

<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>
1:	010100	012345	3
2:	101000	012345	0

Offspring

<i>i</i>	Chromosome <i>abcdef</i>	Unsatisfied clauses	<i>Fitness</i>
1:	110000	012345	3
2:	010100	012345	3
3:	001100	012345	2
4:	101110	012345	1

Old Population

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Termination Criteria

- The evolution (main loop) continues until a **termination (stopping) criterion** is met, possibly until:
 1. A predefined *maximum number of generations* is exceeded
 2. A *goal* is reached, for example:
 1. Expected fitness is achieved
 2. Population converges
 3. Best fitness does not change for a while
 4. A condition is satisfied depending on a combination of above

```
begin
generate initial population; // initialise
calculate fitness values; // evaluate population
do
{
perform reproduction; // select parents
recombine pairs with  $P_c$ ; // apply crossover
apply mutation with  $P_m$ ; // mutate offspring/children
calculate fitness values; // eval. population
replace current population;
} while termination criteria not satisfied;
end
```

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



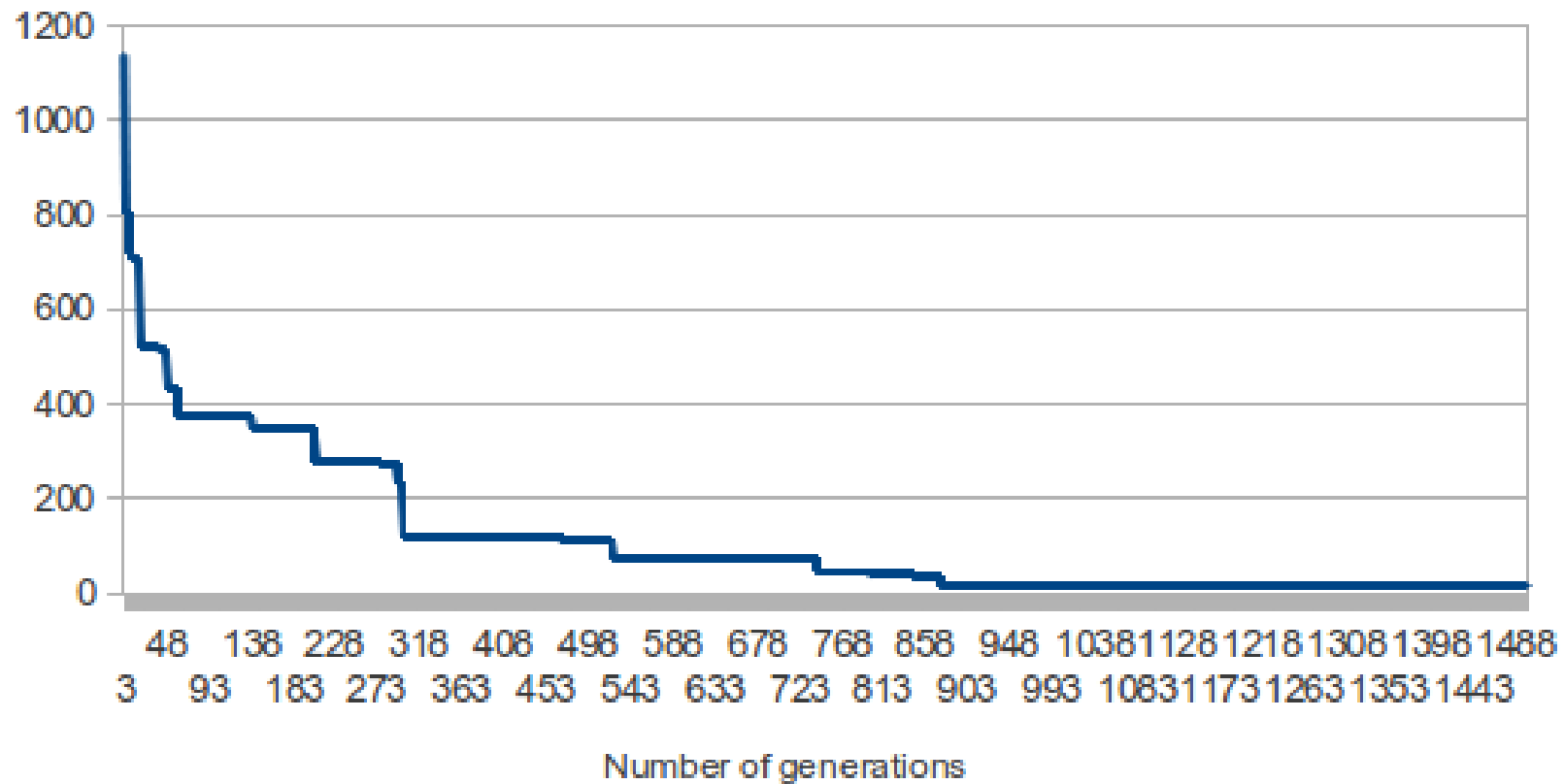
Convergence

- Defined as the progression towards *uniformity* (individuals become alike)
 - **Gene convergence:** a location on a chromosome is converged when 95% of the individuals have the same gene value for that location
 - **Population (Genotypic) convergence:** a population is converged when all the *genes* have converged
 - **Phenotypic Convergence:** *average fitness* of the population approaches to the best individual in the population



Phenotypic Convergence

Score vs Population age



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Summary - Key Features of EAs

- **Population based** search approaches
 - Be independent of initial starting point(s)
 - Start search from many points in the search space
 - Conduct search in *parallel* over the search space
 - implicit parallelism
- Avoid converging to local optima (diversity, exploration)
- Balances exploration and exploitation?
- May be used together with other approaches (hybrids)



University of
Nottingham

UK | CHINA | MALAYSIA

Example:
Optimize $f(x)=x^2$



Example: Optimize $f(x)=x^2$

- **Problem:**
- Find the value of x (0 to 31) that maximizes $f(x)=x^2$
- **Encoding:**
- Represent x as a binary string of 5 bits.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Example: Optimize $f(x)=x^2$

Step 1: Initialize Population

Create a population of 4 random individuals:

- Chromosomes: ["10101", "11100", "00110", "01011"]
- Decode to integers: [21, 28, 6, 11]
- Fitness: $f(x) = x^2$:
 - $f(21) = 441$, $f(28) = 784$, $f(6) = 36$, $f(11) = 121$.



Example: Optimize $f(x)=x^2$

Step 2: Selection

Use **Roulette Wheel Selection** to choose parents:

- Total fitness = $441 + 784 + 36 + 121 = 1382$.
- Selection probabilities:
 - $21 \rightarrow \frac{441}{1382}$
 - $28 \rightarrow \frac{784}{1382}$
 - $6 \rightarrow \frac{36}{1382}$
 - $11 \rightarrow \frac{121}{1382}$

Choose two parents (e.g., 28 and 21).



Example: Optimize $f(x)=x^2$

Step 3: Crossover

Apply single-point crossover:

- Parent 1: "11100", Parent 2: "10101".
- Crossover point: After the 2nd bit.
- Offspring:
 - Child 1: "11101"
 - Child 2: "10100".



Example: Optimize $f(x)=x^2$

Step 4: Mutation

Apply mutation with a small probability (e.g., 0.1):

- Mutate "11101" → "11111" (flip the last bit).
 - Mutate "10100" → "10100" (no change).
-

Step 5: Evaluate Offspring

- Decode:
 - "11111" → $x = 31$, $f(31) = 961$.
 - "10100" → $x = 20$, $f(20) = 400$.
- Replace the weakest individuals in the population.

Step 6: Repeat

Repeat steps 2–5 for 10 generations or until convergence.



Summary - Key Features of EAs

- **Population based** search approaches
 - Be independent of initial starting point(s)
 - Start search from many points in the search space
 - Conduct search in *parallel* over the search space
 - implicit parallelism
- Avoid converging to local optima
- Balances exploration and exploitation?
- May be used together with other approaches (hybrids)

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



University of
Nottingham

UK | CHINA | MALAYSIA

Memetic Algorithm



Memetic Algorithms (MAs)

- Meme (Richard Dawkins): contagious piece of information
- Memes are similar to **local refinement/local search**
- Gene vs meme: **Mememes can change, evolve using rules and time scales** other than the traditional genetic ones
- MAs aim to improve GAs by **embedding local search**



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Memetic Algorithms

- **Memetic Algorithms (MAs)** are a class of evolutionary algorithms that extend Genetic Algorithms (GAs) by *incorporating local search* techniques, such as Hill Climbing.
- MAs incorporate the global exploration capabilities of GAs with the local refinement capabilities of Hill Climbing.
- The goal is to enhance the performance of the algorithm by balancing *exploration* (searching broadly across the solution space) with *exploitation* (refining specific, promising solutions).

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Memetic Algorithms

- MAs make use of **exploration** capabilities of **GAs** and **exploitation** capabilities of **local search**
 - MAs have an explicit mechanism to **balance exploitation and exploration**
- Memetic Algorithms shown to be *much faster and more accurate than GAs on some problems*, and are the “state of the art” on many problems

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Why Hybridize GAs with Hill Climbing?

- Genetic Algorithms are excellent for exploring a wide solution space due to their stochastic nature, but they may converge prematurely to suboptimal solutions (local optima). Hill Climbing, on the other hand, is deterministic and focuses on exploiting a specific region of the search space.
- By hybridizing the two:
 - **GAs** provide the **diversity** needed for a broad search.
 - **Hill Climbing** ensures that the **search intensifies** around promising areas, improving the fine-tuning of solutions.



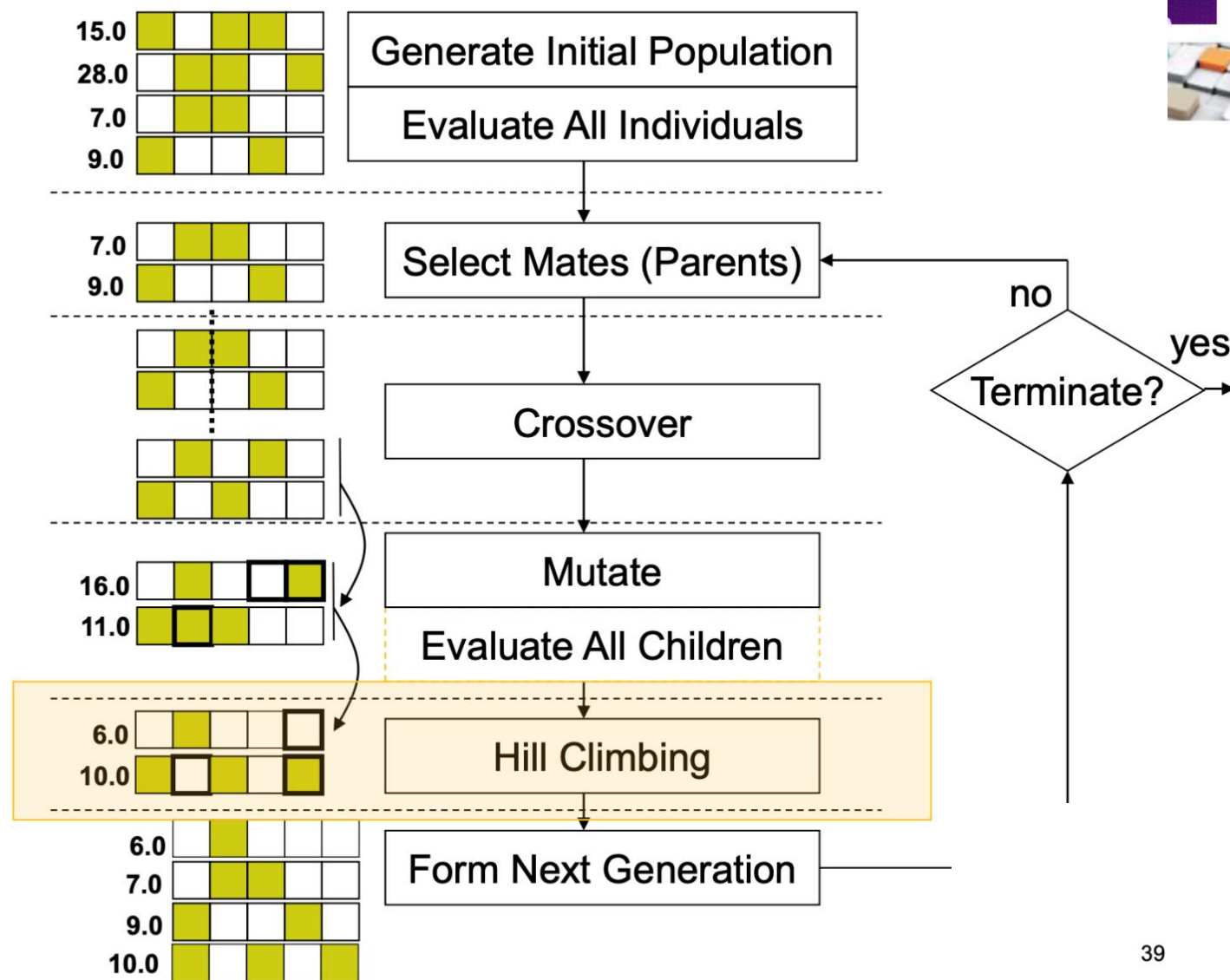
How MAs work?

1. **Initialization:** Start with a population generated as in GAs.
2. **Evolution:** Apply GA operators (selection, crossover, mutation) to evolve the population.
3. **Local Search:** After generating offspring, apply a local search algorithm (like Hill Climbing) to each offspring to refine their solutions.
4. **Replacement/Integration:** Choose which individuals to keep for the next generation, possibly including refined offspring and original individuals. Refined solutions replace less fit individuals in the population, ensuring that the global search benefits from the improved local solutions.
5. **Termination:** The process repeats until a stopping criterion is met (e.g., a maximum number of generations, no significant improvement, or an acceptable fitness level).

ACK: Prof. Ender Özcan, UNILU, Dr. Thomas Maul, UNILU, Dr. Chen ZhiYuan, UNILU



A Generic Memetic Algorithm (MA)



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr C



Memetic Algorithms (MAs)

Pseudocode of memetic algorithm

```
→ CreateInitialSolutions(); // create initial population of solutions

repeat
    → SelectParents(); // select solutions from the population to breed
    → Crossover(); // apply crossover operator with a given probability
    Mutate(); // apply mutation operator with a given probability
    → LocalSearch();
    → Evaluate();
    ReplaceSolutions(); // generate new population of solutions
until TerminationCriteriaSatisfied();
```

Moscato, P.: On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms, Caltech Concurrent Computation Program Report 826, California Institute of Technology (1989)

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Example – Designing a Genetic/Memetic Algorithm for MAX-SAT

1. **Representation:** Bit string of length N (truth assignment for each variable)
2. **Initialisation:** Randomly generate initial population, population size= N
3. **Fitness function:** ($\#$ of clauses $- C$); $\#$ of unsatisfied clauses
4. **Mate selection:** Tournament selection with a tour size of 2
5. **Crossover:** 1PTX, crossover probability = 0.99
6. **Mutation:** random bit-flip, mutation probability = $1/N$
7. **Hill Climbing:** Davis's Bit Hill Climbing
8. **Replacement:** Steady State GA, best two of (parents and offspring) replaces the worst two individuals in the population (strong elitism)

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Example - Designing a Genetic/Memetic Algorithm for TSP

Problem Setup

Suppose we have 6 cities (A, B, C, D, E, F), with the following distance matrix (in km):

	A	B	C	D	E	F
A	0	10	15	20	25	30
B	10	0	35	25	30	20
C	15	35	0	30	20	10
D	20	25	30	0	15	25
E	25	30	20	15	0	10
F	30	20	10	25	10	0

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Example-Designing a Genetic/Memetic Algorithm for TSP

Step 1: Initialization

1. Generate the initial population:

- Create 5 random solutions (routes), where each solution is a permutation of the cities.

Example population:

- Route 1: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow A$
- Route 2: $A \rightarrow F \rightarrow E \rightarrow D \rightarrow C \rightarrow B \rightarrow A$
- Route 3: $A \rightarrow C \rightarrow E \rightarrow B \rightarrow F \rightarrow D \rightarrow A$
- Route 4: $A \rightarrow D \rightarrow B \rightarrow F \rightarrow C \rightarrow E \rightarrow A$
- Route 5: $A \rightarrow E \rightarrow C \rightarrow F \rightarrow B \rightarrow D \rightarrow A$

2. Evaluate fitness:

Calculate the total distance for each route.

Fitness is the inverse of the total distance (smaller distances are better).



Example-Designing a Genetic/Memetic Algorithm for TSP

Step 2: Selection

1. Use a **fitness-proportionate selection** (e.g., roulette wheel) to select parents for reproduction.

Example: Select Route 1 and Route 3 as parents based on their fitness values.

Step 3: Crossover

1. Apply **partially mapped crossover (PMX)** or other crossover techniques to produce offspring.

Example:

- Parent 1: A → B → C → D → E → F → A
- Parent 2: A → C → E → B → F → D → A
- Offspring: A → B → E → D → C → F → A (combination of both parents).

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Example-Designing a Genetic/Memetic Algorithm for TSP

Step 4: Mutation

1. Introduce small random changes to offspring to maintain diversity.

Example: Swap two cities in the route (mutate Offspring to $A \rightarrow B \rightarrow F \rightarrow D \rightarrow C \rightarrow E \rightarrow A$).

Step 5: Local Search (Hill Climbing)

1. Apply Hill Climbing to improve each offspring:

- **Neighborhood exploration:** Swap two adjacent cities, reverse a segment, or relocate a city.
- Evaluate the fitness of each neighboring route and select the best one.

Example for route $A \rightarrow B \rightarrow F \rightarrow D \rightarrow C \rightarrow E \rightarrow A$:

- Swap B and F $\rightarrow A \rightarrow F \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow A$ (new fitness).
- Reverse segment $D \rightarrow C \rightarrow A \rightarrow B \rightarrow F \rightarrow E \rightarrow D \rightarrow A$ (new fitness).
- Select the route with the best fitness.

2. Replace the original offspring with the improved solution.



Example-Designing a Genetic/Memetic Algorithm for TSP

Step 6: Replacement

1. Replace the least fit individuals in the population with the improved offspring.

Example: Replace Route 5 with the locally optimized offspring.

Step 7: Repeat

Repeat Steps 2–6 for a fixed number of generations or until convergence.

Step 8: Output the Best Solution

After the termination condition is met (e.g., 50 generations), output the best route found:

Example: A → E → C → F → B → D → A with a total distance of 70 km.



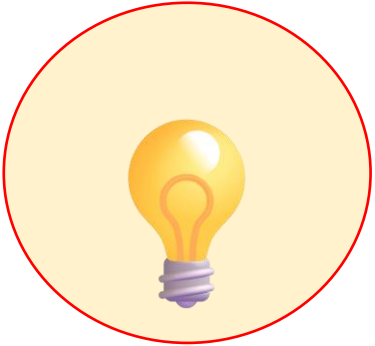
Advantages of Memetic Algorithms

- **Faster Convergence:** By combining global and local search, MAs can converge to high-quality solutions more quickly than traditional GAs.
- **Improved Solution Quality:** Local search enhances the refinement of individuals, leading to better overall solutions.
- **Balance of Exploration and Exploitation:** The GA provides exploration, while Hill Climbing ensures thorough exploitation of good solutions.

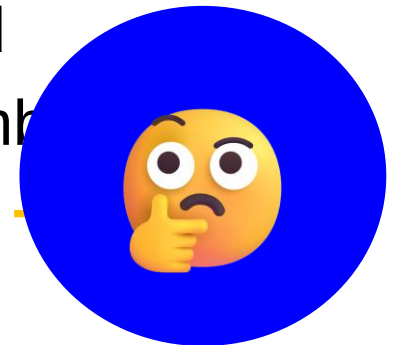
ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



Summary



- **Genetic Algorithms** represent a subclass of nature inspired Evolutionary Computation Techniques
 - GAs creates an initial population of individuals and performs the search by applying selection, crossover and mutation on individuals at each evolutionary cycle
 - GA components including the parameter settings require careful tailoring to the problem in hand
- **Memetic Algorithms** hybridise GAs with Hill Climbing



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



University of
Nottingham

UK | CHINA | MALAYSIA

Questions



Questions

- When does a genetic algorithm perform better than a memetic algorithm?
- Is the choice of *meme* (hill climbing/local search algorithm) important?
- Which meme does have a better performance in a memetic algorithm?
- Can we somehow combine several memes to obtain a synergy leading to improved performance?

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



1. When does a Genetic Algorithm (GA) perform better than a Memetic Algorithm (MA)?

- Genetic Algorithms may outperform Memetic Algorithms in specific scenarios:
- **Exploration-Dominant Problems:**
 - GAs excel when the problem landscape requires extensive exploration and when local optima are not a significant concern. This is because GAs focus on maintaining diversity in the population through genetic operators like crossover and mutation.
- **High-Dimensional or Rugged Landscapes:**
 - In cases where the search space is extremely large or highly rugged (with many local optima), local search methods (used in MAs) may prematurely converge to suboptimal solutions. GAs, with their population-wide search strategy, can better explore such landscapes.



1. When does a Genetic Algorithm (GA) perform better than a Memetic Algorithm (MA)?

- Genetic Algorithms may outperform Memetic Algorithms in specific scenarios:
- **Time-Constrained Scenarios:**
 - MAs involve computational overhead due to local search on individuals, which can slow them down. In problems where time is a critical factor, GAs might be preferred for their faster iteration cycles.
- **Sparse Problem Knowledge:**
 - If no reliable local search (meme) exists for a problem, GAs may perform better since they don't rely on problem-specific heuristics.



2. Is the choice of meme (hill climbing/local search algorithm) important?

- Yes, the choice of meme is critical in a Memetic Algorithm. The effectiveness of the MA heavily depends on how well the local search strategy matches the problem characteristics. Key considerations include:
 - **Problem Landscape:**
 - A smooth landscape may benefit from gradient-based search, while rugged landscapes may require stochastic or heuristic-based local search methods.
 - **Convergence Speed:**
 - Some memes (e.g., hill climbing) are fast but can easily get stuck in local optima. Others (e.g., simulated annealing) can escape local optima but may take longer to converge.
 - **Exploration vs. Exploitation:**
 - The meme should balance exploration (broad search) and exploitation (intensified search in promising areas). Poorly chosen memes can lead to premature convergence or wasted computational effort.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



3. Which meme has better performance in a Memetic Algorithm?

- There is no universally "best" meme; performance depends on the specific problem. However, some general insights include:
 - **Hill Climbing:**
 - Performs well for problems with unimodal or relatively smooth fitness landscapes. It is fast and simple but prone to getting stuck in local optima.
 - **Simulated Annealing:**
 - Useful in multimodal landscapes because of its ability to escape local optima. However, it is computationally expensive.



3. Which meme has better performance in a Memetic Algorithm?

- **Tabu Search:**
 - Effective for problems with repeated solutions or cyclical search patterns due to its memory-based mechanism.
- **Greedy Search:**
 - Works well in structured problems where small, incremental improvements lead to optimal solutions.
- **Problem-Specific Heuristics:**
 - Tailored search algorithms often outperform general-purpose memes but require deep problem knowledge.



4. Can we combine several memes to obtain synergy leading to improved performance?

- Yes, combining multiple memes can lead to synergistic improvements. This approach is often referred to as **Multimeme Memetic Algorithms** or **Hybrid Memetic Algorithms**. Techniques include:
 - **Dynamic Meme Selection:**
 - Allow individuals in the population to switch between memes based on their performance. For instance, use hill climbing for early stages and simulated annealing for escaping local optima.
 - **Co-evolution of Memes:**
 - Evolve not just solutions but also the associated memes. Memes that perform well are propagated, while less effective ones are discarded.



4. Can we combine several memes to obtain synergy leading to improved performance?

- **Parallel Meme Application:**
 - Apply different memes to subpopulations or individuals in parallel, enabling diverse search strategies to explore the problem space.
- **Meme Ensembles:**
 - Use an ensemble of memes for each individual, where a weighted combination or a sequence of memes is applied.
- **Adaptive Meme Tuning:**
 - Adjust meme parameters dynamically based on the problem landscape, individual progress, or feedback from fitness evaluations.



University of
Nottingham

UK | CHINA | MALAYSIA

Questions



University of
Nottingham

UK | CHINA | MALAYSIA

NEXT:

Evolutional Algorithms II